

COMP90050 ADBS

Week 8

Q1 Deadlock

1. If we use the following comments to lock and unlock access to objects then: Which transactions below are in deadlock if they start around the same time?

T1	T2	T3	T4
Begin	Begin	Begin	Begin
LOCK(C)	LOCK(A)	LOCK(C)	LOCK(B)
Write C	Write A	Write C	Write B
UNLOCK(C)	LOCK(B)	UNLOCK(C)	LOCK(A)
End	Write(B)	End	Write A
	UNLOCK(A)		UNLOCK(A)
	UNLOCK(B)		UNLOCK(B)
	End		End

Deadlock: each process in the deadlock is waiting for another member to release the resources it wants.

T2 and T4 are in a deadlock as each of them will wait for the other to release a lock while holding a lock that the other needs to acquire to complete.

Q2 Dependency

2. What are the dependencies in the following history (a sequence of tuples in the form (T_i, O_i, T_j))? Draw the dependency graph mapping to this dependency set as well.

$H = \langle (T1, R, O1), (T3, W, O5), (T3, W, O1), (T2, R, O5), (T2, W, O2), (T5, R, O4), (T1, R, O2), (T5, R, O3) \rangle$

Dependency

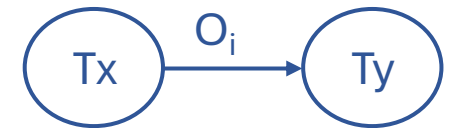
Ty depends on Tx if:

- Tx performs an action on an object O_i , then Ty performs an action on the same O_i , AND
- there is no write action in between by another transaction on O_i

(Do not focus on READ-READ dependency)

Dependency: (Tx, O_i, Ty)

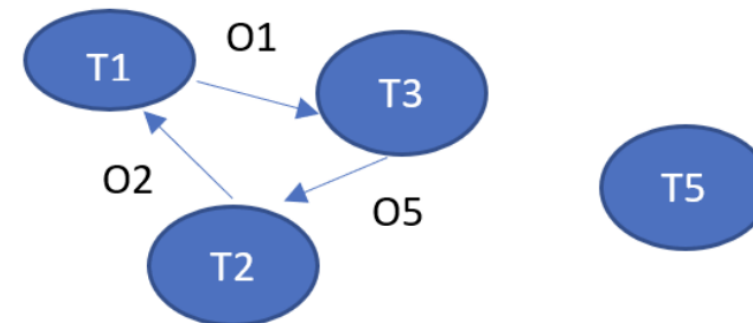
Dependency graph:



Dependency graph

Transactions are nodes, and object labels the edges from the node Tx to Ty if dependency (Tx, O, Ty) exists.

$DEP(H) = \{ \langle T1, O1, T3 \rangle, \langle T3, O5, T2 \rangle, \langle T2, O2, T1 \rangle \}$



Q3 Wormhole

3. Given the solution above for question 2, can we say the history is equal to a serial history. If yes, show one such history. If not, show that there is a wormhole.

Serial history: a history resulted from running transactions sequentially one by one

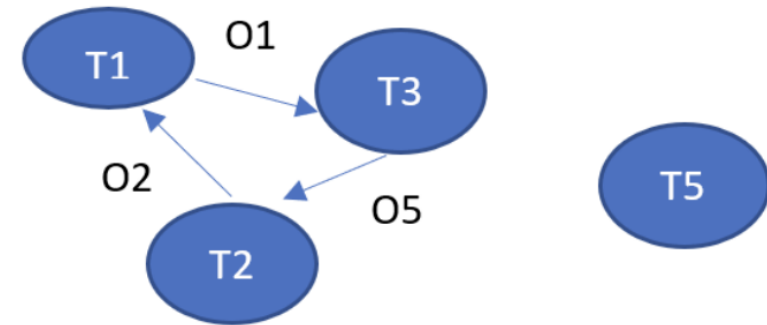
Equal histories: two histories are equal if their dependencies contain the same tuples, which implies that the database will end up in exactly the same final state

A transaction T' is called a **wormhole transaction** if

$$T' \in Before(T) \cap After(T)$$

That is $T \ll T' \ll T$. This implies there is a cycle in the dependency graph of the history. Presence of a wormhole transaction implies it is not isolated ($\Rightarrow$ not a serial schedule).

Solution for Q2:



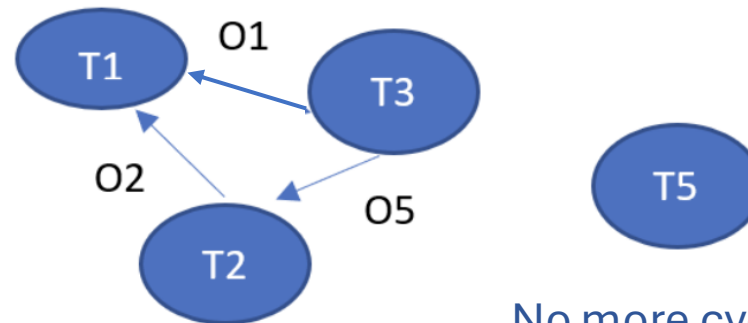
- There exists a cycle in the dependency graph, so **there are wormhole transactions**
 - E.g., T1 is before and after of T3 at the same time
- Therefore finding equivalent serial execution is not possible.

Q4 Dependency & Wormhole

4. Repeat questions 2 and 3 with history: Find dependencies and draw dependency graph, determine if equal to a serial history (show one such history) or if wormholes exist

$H = \langle (T3, W, O5), (T3, W, O1), (T2, R, O5), (T2, W, O2), (T5, R, O4), (T1, R, O1), (T1, R, O2), (T5, R, O3) \rangle$

$DEP(H) = \{ \langle T3, O5, T2 \rangle, \langle T3, O1, T1 \rangle, \langle T2, O2, T1 \rangle \}$



No more cycles in the dependency graph!

Equal to e.g. serial history of **serial execution of T3, T2, T1, T5**:

$H = \langle (T3, W, O5), (T3, W, O1), (T2, R, O5), (T2, W, O2), (T1, R, O1), (T1, R, O2), (T5, R, O4), (T5, R, O3) \rangle$

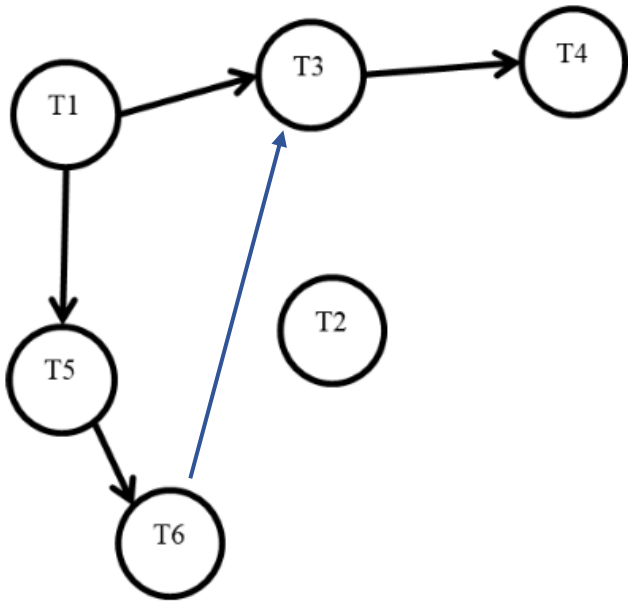
The DEP of this history is the same as the DEP of the history given in the question

Q5 Wormhole

5. Given the graph below for six transactions we can see the dependency between the transactions. If we add a link from T6 to T3, the graph becomes more complicated. Please in that case list $\text{After}(T6)$ and $\text{Before}(T6)$. Does this link cause a wormhole to appear, in otherwise a set of dependencies where there were no wormholes in the original version of the graph? Instead, if we have a link added from T4 to T1, please list $\text{After}(T4)$ and $\text{Before}(T4)$. Does this link cause a wormhole? Which one of the two modifications represents dependencies for a history that we can call isolated?

Wormhole theorem: A history is isolated if and only if it has no wormholes.

(History is **isolated** if equivalent to a serial history)



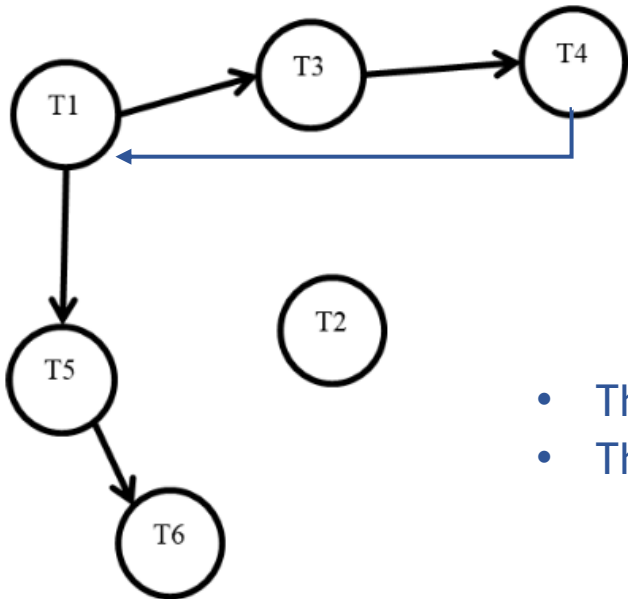
$\text{After}(T6) = \{ T3, T4 \}$

$\text{Before}(T6) = \{ T1, T5 \}$

- $\text{After}(T6)$ has no intersection with $\text{Before}(T6)$, thus by adding the link we did not create a cycle (no wormhole transaction created)
 - If a cycle was formed then T6 would have been a part of it and would create a wormhole transaction.
- As the original graph did not have a wormhole and this new version does not create a wormhole transaction as well, then this modification's **dependencies come from an isolated history**.

Q5 Wormhole

5. Given the graph below for six transactions we can see the dependency between the transactions. If we add a link from T6 to T3, the graph becomes more complicated. Please in that case list $\text{After}(\text{T6})$ and $\text{Before}(\text{T6})$. Does this link cause a wormhole to appear, in otherwise a set of dependencies where there were no wormholes in the original version of the graph? Instead, if we have a link added from T4 to T1, please list $\text{After}(\text{T4})$ and $\text{Before}(\text{T4})$. Does this link cause a wormhole? Which one of the two modifications represents dependencies for a history that we can call isolated?



$\text{After}(\text{T4}) = \{ \text{T1}, \text{T3}, \text{T5}, \text{T6} \}$

$\text{Before}(\text{T4}) = \{ \text{T1}, \text{T3} \}$

- Their intersection is not an empty set and e.g. T1 becomes a wormhole
- Thus, the history that leads to that set of dependencies cannot be an isolated history.

Q6 Two-phase Locking

6. Given the operations for a transaction T1 below, please list the lines that this transaction is executing that cannot happen with two-phase locking. Briefly explain.

```
1  Slock(A)
2  Read(A)
3  Unlock(A)
4  Slock(B)
5  Read(B)
6  Unlock(B)
7  Xlock(C)
8  Write(C)
9  Unlock(C)
10 Xlock(A)
11 Write(A)
12 Unlock(A)
```

Two phase transactions: A transaction is two phased if all LOCK operations precede all its UNLOCK operations.

- **Lines 4, 7, and 10** cannot happen because the first unlock operation is in Line 3.
- There cannot be a lock operation after the start of the shrinking phase, i.e., the transaction cannot get further locks after the first unlock operation.